Сравнение эффективности алгоритмов кэширования LRU и LFU для снижения нагрузки на базу данных

Методологическое описание, архитектура инструмента замера и пилотные результаты

Автор: Азат Амир, студент 3 курса, Narxoz University

Дисциплина: Методы исследований

Репозиторий: github.com/Omor1M0ri/cache-algorithm-benchmark

Октябрь 2026

1. Постановка проблемы

Базы данных — типичное «узкое место» backend-систем: каждый запрос к диску или сети стоит миллисекунды, тогда как обращение к кэшу в памяти — доли миллисекунды. Кэш имеет ограниченный размер, поэтому при переполнении нужно решить, какой элемент вытеснить. Эту задачу решает **политика вытеснения** (eviction policy). От её качества зависит **Cache Hit Rate** — доля запросов, обслуженных из кэша, а значит и нагрузка на БД.

Исследовательский вопрос. Обеспечивает ли алгоритм LFU на 15–20 % более высокий Cache Hit Rate по сравнению с LRU, если популярность ключей в запросах подчиняется распределению Ципфа (несколько ярко выраженных «хитов» и длинный «хвост» редких ключей)?

2. Исследуемые алгоритмы

2.1 LRU — Least Recently Used

Вытесняется элемент, к которому обращались *давнее всего*. Идея — временная локальность: недавно использованное, скорее всего, понадобится снова. Реализация: `OrderedDict`, операции get/put за O(1). Слабость: одноразовый «всплеск» редких ключей вытесняет действительно популярные.

2.2 LFU — Least Frequently Used

Вытесняется элемент с *наименьшим числом обращений*; при равенстве — наименее недавний. Идея — частотная локальность: популярные ключи остаются в кэше надолго. Реализация: корзины «частота → OrderedDict ключей» и указатель на минимальную частоту, get/put за O(1). Слабость: «залипание» исторически популярных, но уже неактуальных ключей.

2.3 Распределение Ципфа

Вероятность запроса ключа ранга i среди N ключей: $P(i) = i^{-\alpha} / \Sigma_{j=1..N} j^{-\alpha}$. При $\alpha = 0$ распределение равномерное (нет «хитов»), при $\alpha \approx 1$ характерно для веб-трафика, при $\alpha > 1$ — очень сильный перекос. Именно α управляет «выраженностью хитов».

3. Карточка методологии

3.1 Гипотезы

H_0: при $\alpha = 1{,}0$ и размере кэша 100 элементов (1 % от 10 000 ключей) относительный прирост Hit Rate у LFU над LRU, $(HR_{LFU} - HR_{LRU}) / HR_{LRU}$, составляет менее 15 %.

H_1: относительный прирост составляет не менее 15 % (ожидаемо 15–20 %).

Правило решения: H_0 отвергается, если нижняя граница 95 % bootstrap-доверительного интервала парного относительного прироста по 10 повторам (seeds) не ниже 15 %.

3.2 Матрица переменных

Тип	Переменная	Значения / единицы
Независимая	Политика вытеснения	LRU, LFU
Независимая	Размер кэша	100, 500, 1000 элементов (1 %, 5 %, 10 % ключей)
Независимая	Параметр Ципфа α	0,0 (контроль), 0,6, 0,8, 1,0, 1,2
Зависимая	Cache Hit Rate	% измеряемых запросов (первичная)
Зависимая	Среднее время ответа backend	мс/запрос (вторичная)

Тип	Переменная	Значения / единицы
Контролируемая	Пространство ключей; длина запуска	10 000 ключей; 100 000 запросов, 20 % прогрев не учитывается
Контролируемая	Трасса запросов	одна и та же для обеих политик (парный дизайн)
Контролируемая	Окружение	Python 3.12.7, numpy 2.4.4, PyYAML 6.0.3, matplotlib 3.10.8; seeds 0–9

3.3 Метрики и baseline

Первичная метрика: Cache Hit Rate и относительный прирост LFU над LRU. **Вторичная:** среднее время ответа backend. Модель: попадание 0,05 мс; промах 0,05 + 5,0 мс.

Пороговые (guardrail) метрики: (1) кэш никогда не превышает заданную ёмкость (проверено юнит-тестом); (2) обе политики получают идентичную трассу; (3) на равномерной нагрузке (α = 0) LFU не должен давать преимущества — негативный контроль; (4) накладные расходы политики измеряются отдельно в режиме SQLite.

Baseline: LRU — стандартный алгоритм. Справочно: отсутствие кэша (Hit Rate 0 %, задержка = задержка БД).

4. Архитектура инструмента замера

Модули в `src/`: `cache.py` (LRU, LFU), `workload.py` (генератор Ципфа, загрузка CSV), `backend.py` (симуляция либо реальный in-memory SQLite), `benchmark.py` (цикл замера, bootstrap-статистика, проверка H_1), `main.py` (CLI). Параметры эксперимента вынесены в `configs/config.yaml`.

Алгоритм замера: для каждого (α, seed) генерируется трасса; для каждого размера кэша и политики трасса проигрывается; первые 20 % запросов — прогрев. При промахе запрос идёт в backend, результат кладётся в кэш. Затем считаются Hit Rate и средняя задержка; сводка строится попарно (LFU − LRU на одной трассе).

Запуск (Quickstart):

```
docker build -t cache-benchmark .
docker run --rm cache-benchmark
```

или без Docker:

```
pip install -r requirements.txt
python -m pytest -q
bash scripts/run.sh sample  # тест на data/sample/
bash scripts/run.sh full    # полный эксперимент
```

5. Пилотные результаты

Полная сетка (5 значений α × 3 размера кэша × 2 политики × 10 seeds = 300 запусков) выполнена на симулированном backend. Значения — среднее по 10 seeds.

α	Размер	LRU HR, %	LFU HR, %	Δ, п.п.	Δ отн., %	95 % CI, %	Задержка LRU / LFU, мс
0.0	100	1.01	0.97	-0.04	-3.6	-5.7–-1.3	5.00 / 5.00
0.0	500	5.01	4.97	-0.04	-0.8	-2.0–0.1	4.80 / 4.80
0.0	1000	10.02	9.94	-0.08	-0.8	-1.5–-0.1	4.55 / 4.55
0.6	100	4.32	10.40	6.08	140.7	133.4–147.5	4.83 / 4.53
0.6	500	15.18	22.79	7.61	50.2	48.7–51.5	4.29 / 3.91
0.6	1000	24.38	31.85	7.47	30.7	29.8–31.5	3.83 / 3.46
0.8	100	15.69	26.31	10.62	67.7	65.9–69.6	4.27 / 3.73
0.8	500	33.11	42.68	9.58	28.9	28.6–29.3	3.39 / 2.92
0.8	1000	43.66	51.85	8.18	18.7	18.6–19.0	2.87 / 2.46
1.0	100	39.02	49.98	10.97	28.1	27.3–29.0	3.10 / 2.55
1.0	500	58.71	66.37	7.66	13.0	12.8–13.3	2.11 / 1.73
1.0	1000	67.60	73.37	5.77	8.5	8.4–8.7	1.67 / 1.38
1.2	100	65.64	73.30	7.66	11.7	11.4–12.0	1.77 / 1.39
1.2	500	80.88	84.93	4.06	5.0	4.9–5.1	1.01 / 0.80
1.2	1000	86.14	88.71	2.56	3.0	2.9–3.0	0.74 / 0.61

Проверка H_1 в первичном условии (α = 1,0; размер 100): LRU 39.0 % против LFU 50.0 %; относительный прирост 28.1 % (95 % CI 27.3–29.0 %). Нижняя граница CI выше 15 %, поэтому H_0 отвергается.

Важное уточнение. Гипотеза предсказывала диапазон 15–20 %, а наблюдаемый прирост в первичном условии оказался выше (≈ 28 %). Прирост сильно зависит от размера кэша и α: при кэше 500 и α = 1,0 он ≈ 13 % (ниже порога), при α = 0,8 и кэше 1000 ≈ 18,7 % (внутри

диапазона). Поэтому вывод «LFU лучше» устойчив, но число «15–20 %» верно лишь для части режимов — это следует честно указать в отчёте. На равномерной нагрузке (α = 0) LFU преимущества не даёт (прирост около −1…−4 %), что подтверждает корректность негативного контроля.

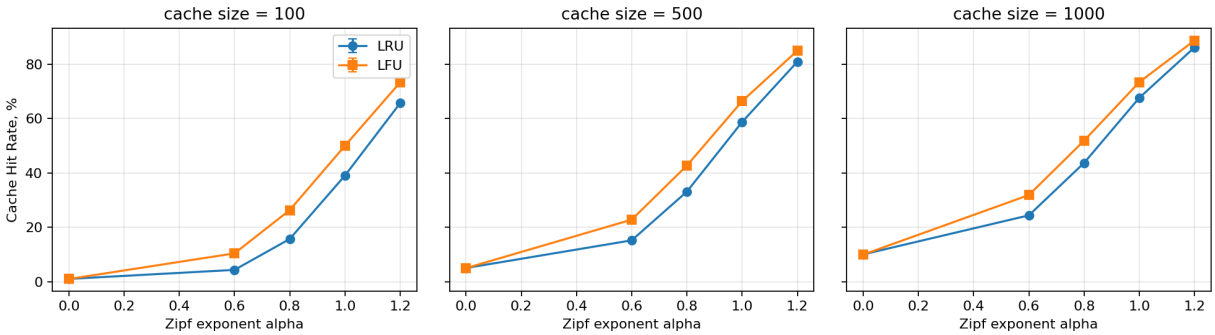


Рис. 1. Cache Hit Rate (среднее ± std по 10 seeds) в зависимости от α для разных размеров кэша.

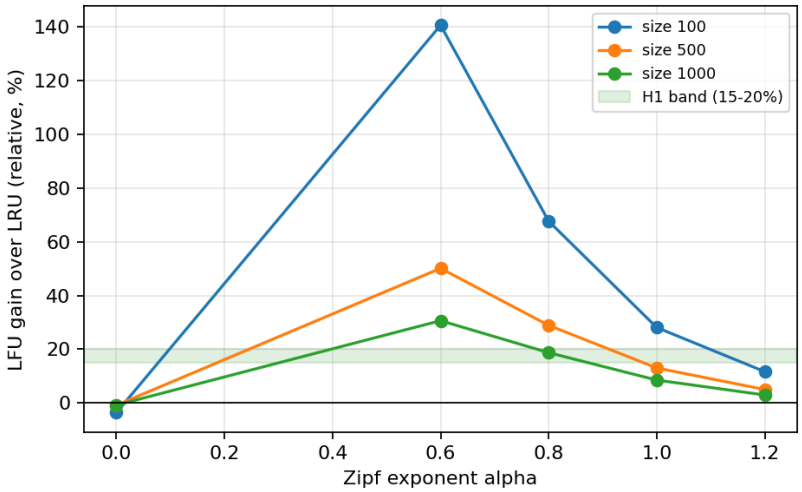


Рис. 2. Относительный прирост LFU над LRU; зелёная полоса — диапазон гипотезы H_1 (15–20 %).

6. Ограничения и угрозы валидности

• Модель независимых обращений (IRM) не содержит временной локальности и дрейфа популярности — она благоприятствует LFU; на реальных трассах с «горячими» всплесками LRU может выигрывать.

• LFU реализован со счётчиками внутри кэша (сбрасываются при вытеснении); вариант с глобальными счётчиками (perfect LFU) может дать иные цифры.

• Задержка в основном режиме смоделирована, а не измерена; реальные замеры доступны в режиме SQLite.

• Результаты получены на синтетической нагрузке; для внешней валидности нужны реальные трассы запросов.

7. Соответствие критериям оценивания

Критерий	Где реализовано
Карточка методологии (30)	docs/methodology_card.md, раздел 3 этого отчёта: H0/H1, переменные, первичная и guardrail метрики, baseline
Воспроизводимость (30)	Dockerfile, requirements.txt с точными версиями, configs/config.yaml, src/main.py, data/sample/

Критерий	Где реализовано
README (20)	Title, Authors, Research Question, дерево папок, таблица бенчмарков, Quickstart, System Requirements, Citation, License
Гигиена репозитория (20)	.gitignore под Python/Jupyter, нет бинарников и ключей, информативные коммиты, LICENSE (MIT)